

Redes Neurais e Deep Learning

Prof. Denilson Alves Pereira

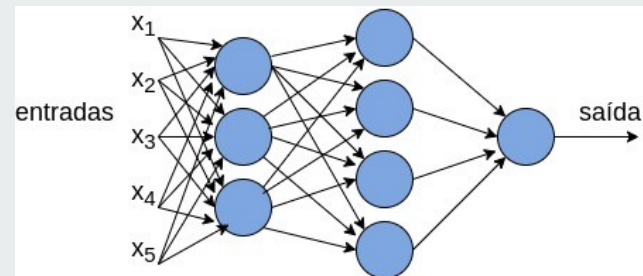
denilsonpereira@ufla.br

<https://sites.google.com/ufla.br/denilsonpereira/>

Universidade Federal de Lavras

Instituto de Ciência Exatas e Tecnológicas

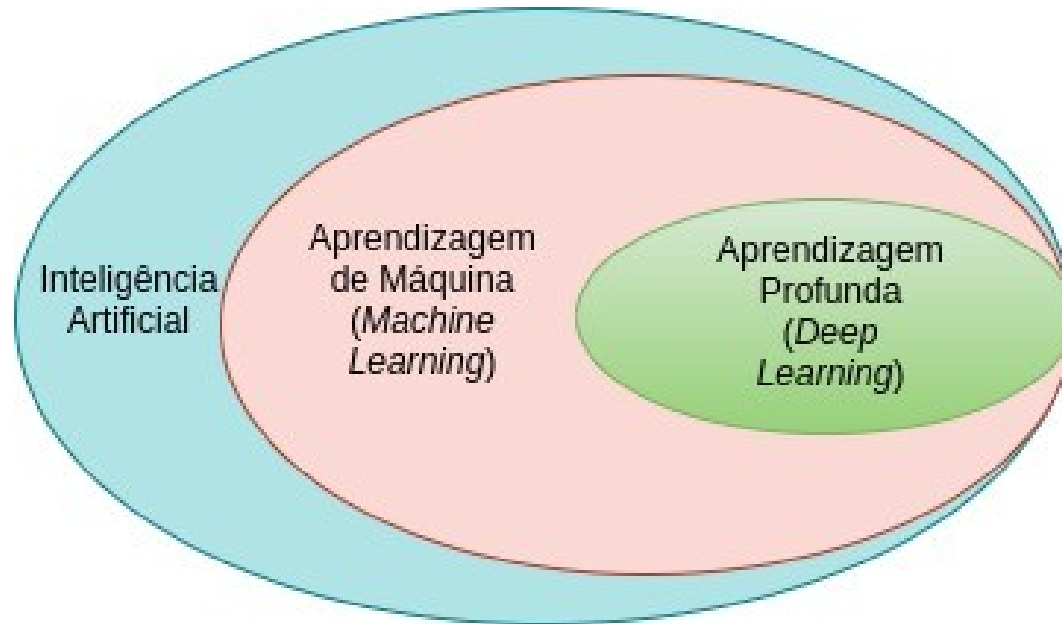
Departamento de Ciência da Computação



Conteúdo



- ◆ Introdução às Redes Neurais Artificiais
- ◆ Deep Learning
- ◆ O Modelo Perceptron
- ◆ Função de Ativação
- ◆ Descida do Gradiente
- ◆ *Backpropagation* (retropropagação)



Machine Learning



Tarefas difíceis de serem programadas:

- ◆ Reconhecer objetos tridimensionais sob diferentes ângulos e iluminação
 - ◆ não sabemos como nosso cérebro faz isso
- ◆ Identificar fraudes
 - ◆ enumerar um conjunto de regras não é simples, e o problema é dinâmico

Machine Learning



Solução:

Usar técnicas de machine learning para aprender modelos com os dados

Dados, dados, dados ... são a chave para machine learning

Machine Learning



Algoritmos aprendem padrões em dados

Padrões aprendidos permitem fazer previsões relativas a novos dados

Machine Learning



Dois tipos principais de técnicas de aprendizagem:

- ◆ Aprendizagem supervisionada
 - ◆ Dados para treinamentos fornecidos como entrada
- ◆ Aprendizagem não-supervisionada
 - ◆ Nenhum dado para treinamento fornecido

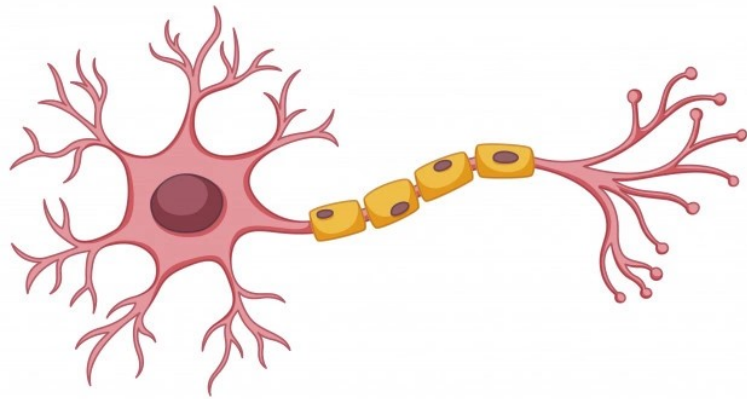
Machine Learning



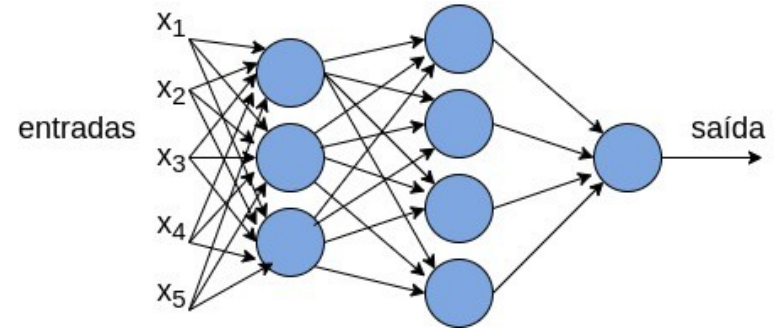
Alguns métodos clássicos de machine learning:

- ◆ Support Vector Machine (SVM)
- ◆ Naive Bayes
- ◆ Árvore de Decisão
- ◆ Knn
- ◆ K-Means

Redes Neurais



Neurônio Biológico



Rede de Neurônios Artificiais

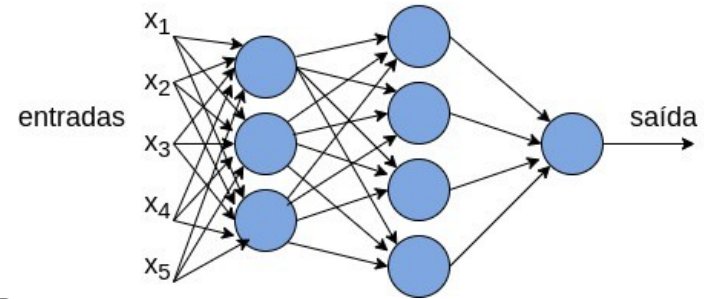
Redes Neurais



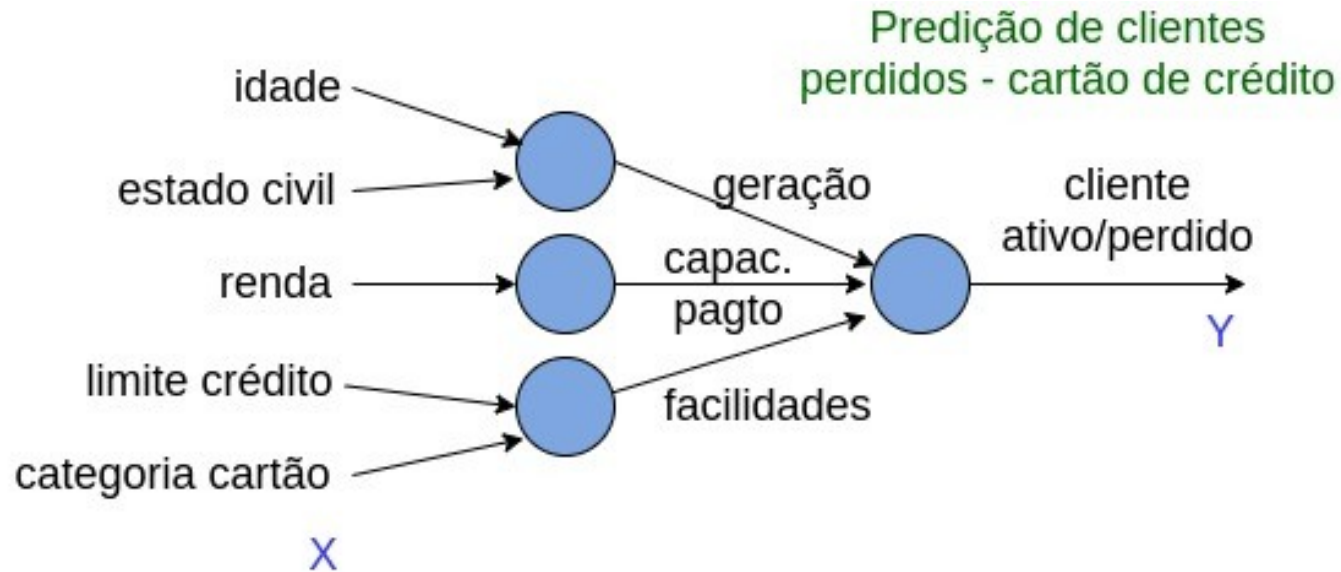
- ◆ Existem desde a década de 1950
- ◆ Se popularizaram nas últimas duas décadas
 - ◆ Volume de dados disponível (Big Data)
 - ◆ Maior capacidade de processamento
 - ◆ Processamento paralelo usando GPUs
 - ◆ Vetores e matrizes

Deep Learning

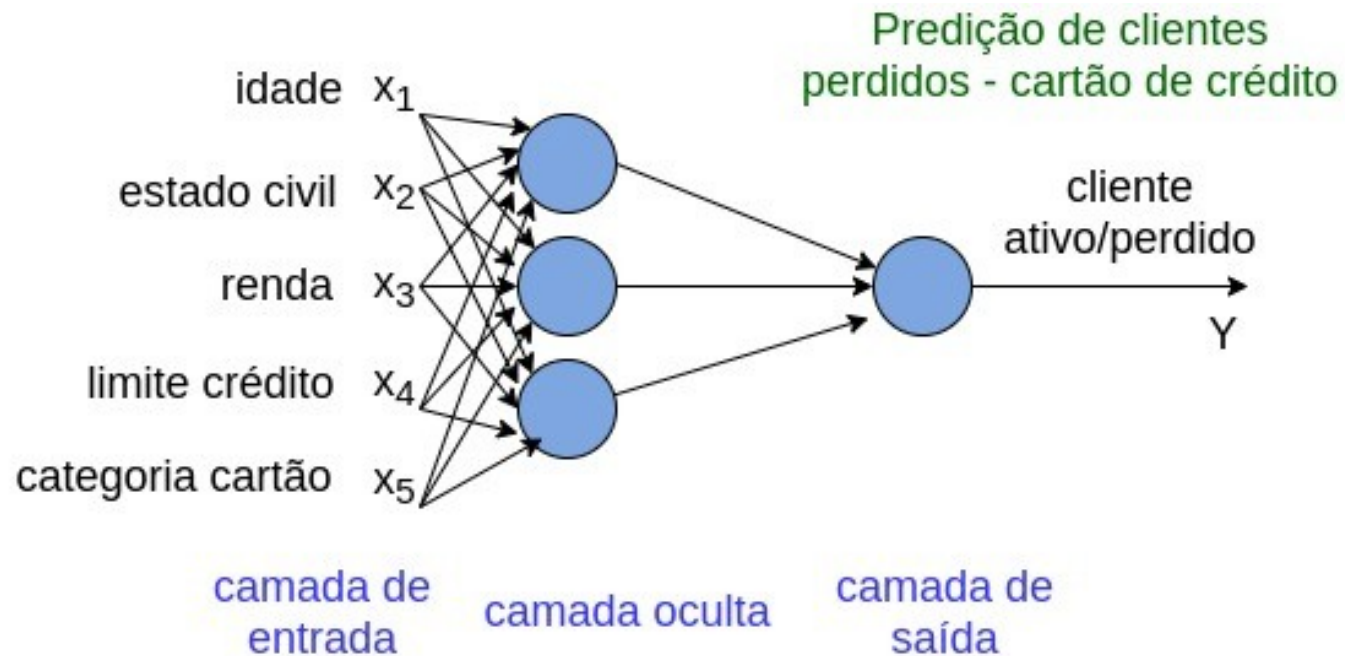
- ◆ Aprendizagem profunda
 - ◆ Modelos matemáticos complexos
- ◆ Usa rede neurais profundas, com diversas camadas



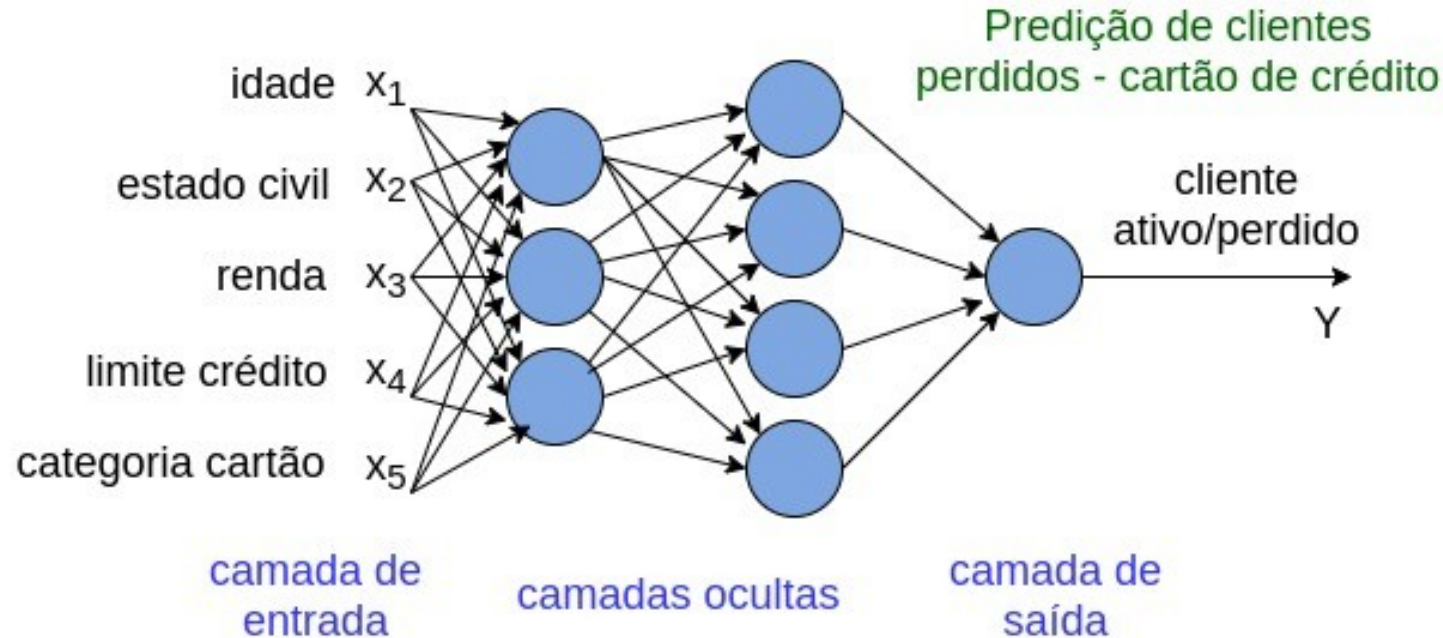
Rede Neural



Rede Neural



Rede Neural



Aplicações de Redes Neurais



Aplicação	Entrada (X)	Saída (Y)
Clientes cartão de crédito	Atributos cliente/cartão	Ativo? (sim/não)
Imobiliária	Atributos do imóvel	Preço
Detecção de objetos	Image	Objeto (1, 2, ..., n)
Reconhecimento de voz	Áudio	Texto transcrito
Tradução automática	Texto em Inglês	Texto em Português
Veículo autônomo	Image, dados sensores	Posição de outros carros

Arquiteturas de Redes Neurais

- ◆ Perceptron, Feed Forward, Radial Basis
- ◆ Convolutional Neural Network (CNN)
- ◆ Recurrent Neural Network (RNN)
- ◆ Long/Short Term Memory (LSTM)
- ◆ Generative Adversarial Network (GAN)
- ◆ . . .
- ◆ <https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464>



O Modelo Perceptron

Entrada/Saída de Dados

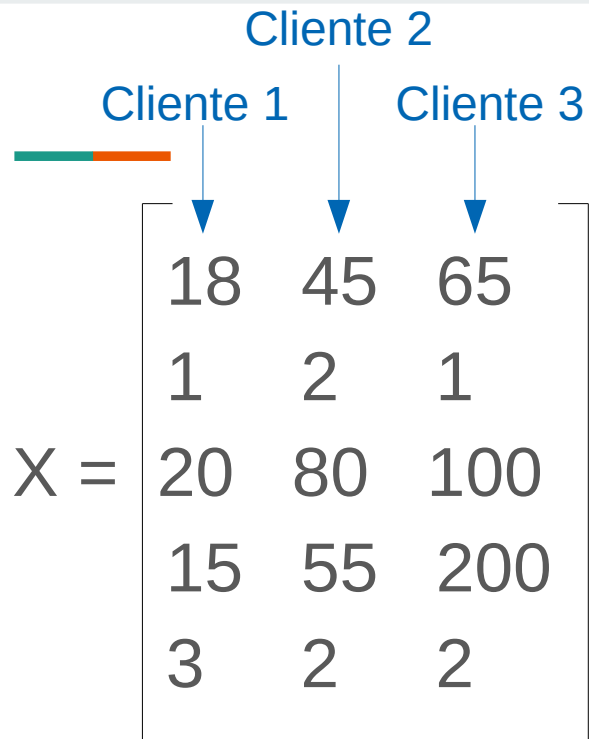
Vetor de atributos (*features*)

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ x_{31} & x_{32} & \dots & x_{3m} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix}$$

$$Y = [y_{11} \ y_{12} \ \dots \ y_{1m}]$$

linha = atributo (n atributos)

coluna = exemplo de treino/teste
(m exemplos)



	Cliente 1	Cliente 2	Cliente 3
$X =$	18	45	65
	1	2	1
	20	80	100
	15	55	200
	3	2	2

← idade

← estado civil

← renda

← limite de crédito

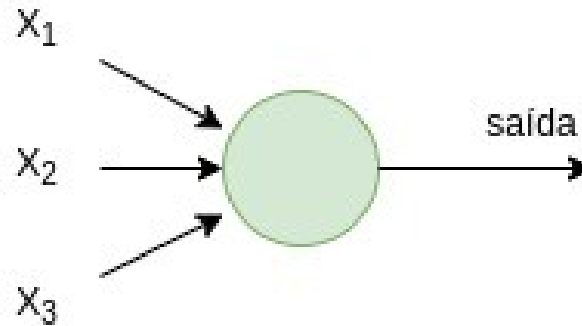
← categoria cartão

$$Y = [0 \quad 1 \quad 1]$$

← 0 – perdido, 1 – ativo

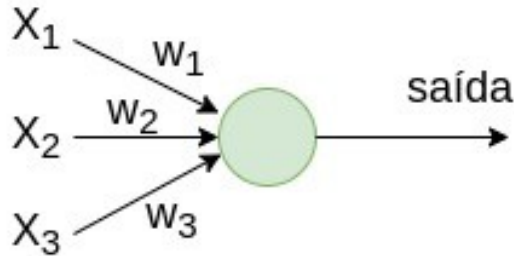
Perceptron

É um modelo matemático que recebe várias entradas $x_1, x_2, \dots$ e produz uma única saída binária



Perceptron

$$\text{saída} = \begin{cases} 0 & \text{se } \sum_j x_j * w_j \leq \text{threshold} \\ 1 & \text{se } \sum_j x_j * w_j > \text{threshold} \end{cases}$$



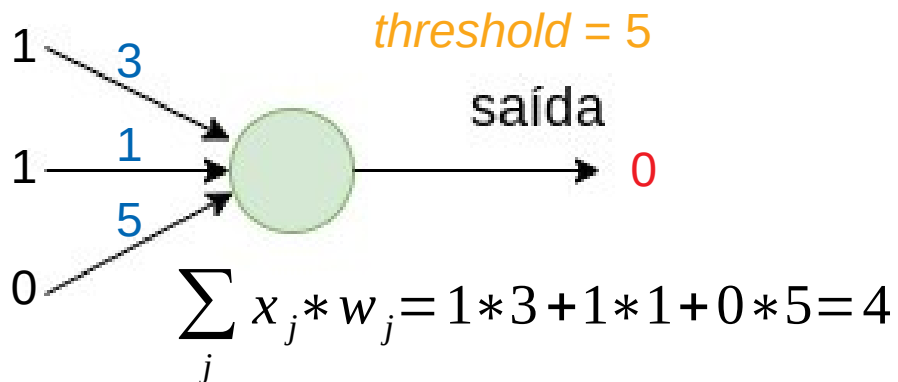
w_j = peso para cada entrada j
expressando sua importância

threshold = limiar

Ambos são parâmetros do
neurônio (números reais)

Perceptron

$$\text{saída} = \begin{cases} 0 & \text{se } \sum_j x_j * w_j \leq \text{threshold} \\ 1 & \text{se } \sum_j x_j * w_j > \text{threshold} \end{cases}$$



Você joga tênis hoje?

$x_1 = 1$ (tempo bom)

$w_1 = 3$

$x_2 = 1$ (está ventando)

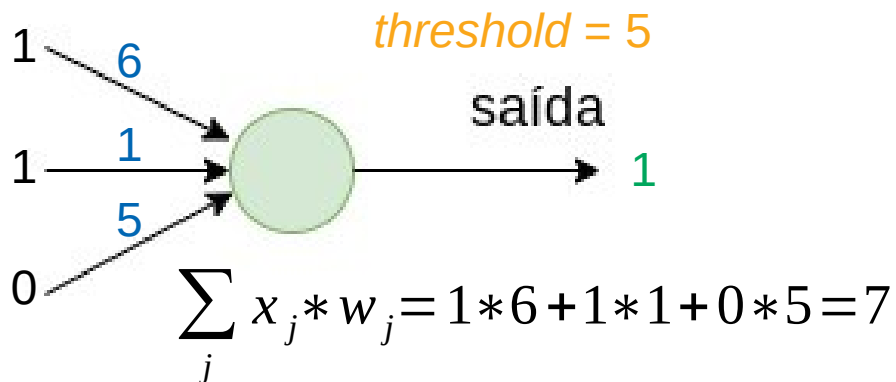
$w_2 = 1$

$x_3 = 0$ (estou de bom humor)

$w_3 = 5$

Perceptron

$$\text{saída} = \begin{cases} 0 & \text{se } \sum_j x_j * w_j \leq \text{threshold} \\ 1 & \text{se } \sum_j x_j * w_j > \text{threshold} \end{cases}$$



Você joga tênis hoje?

$x_1 = 1$ (tempo bom)

$w_1 = 6$

$x_2 = 1$ (está ventando)

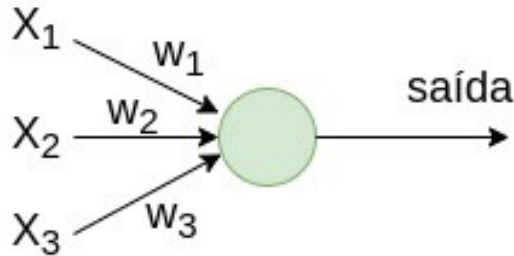
$w_2 = 1$

$x_3 = 0$ (estou de bom humor)

$w_3 = 5$

Perceptron

$$\text{saída} = \begin{cases} 0 & \text{se } w^T x + b \leq 0 \\ 1 & \text{se } w^T x + b > 0 \end{cases}$$



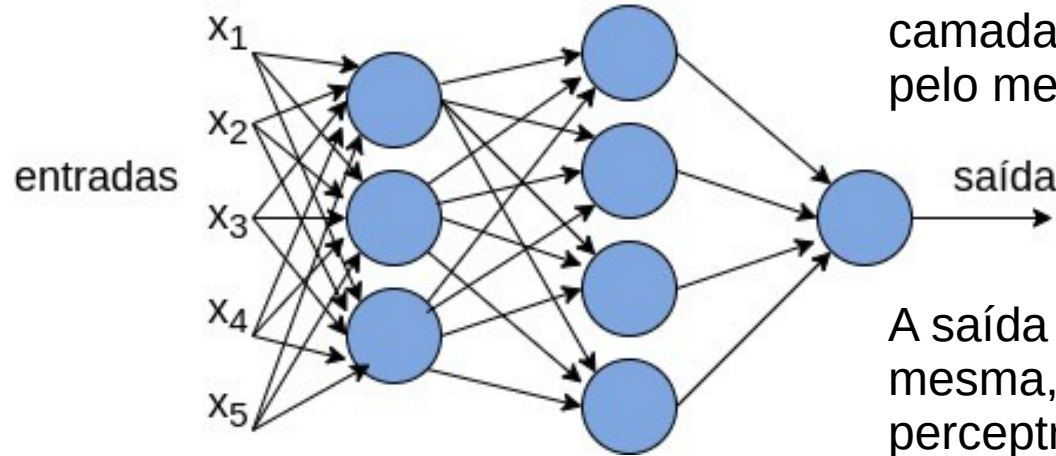
Notação simplificada

w e x são vetores de pesos e entradas, respectivamente

w^T = vetor transposto de w

b = viés (b = -threshold)

Multilayer Perceptron

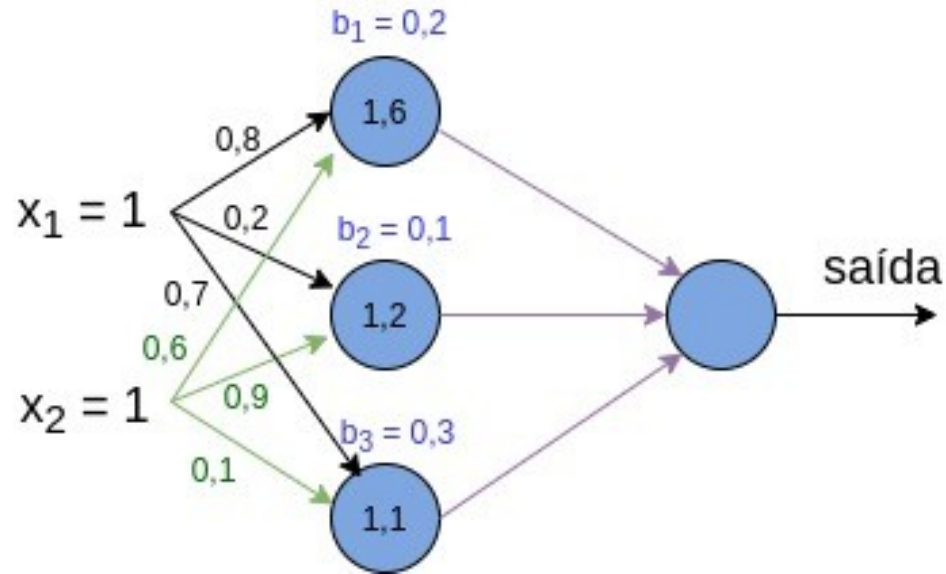


Rede de perceptrons de várias camadas (*Multilayer Perceptrons*), pelo menos duas camadas

A saída de cada perceptron é a mesma, e é replicada para os perceptrons da camada seguinte

Fully connected network (rede completamente conectada)

Multilayer Perceptron



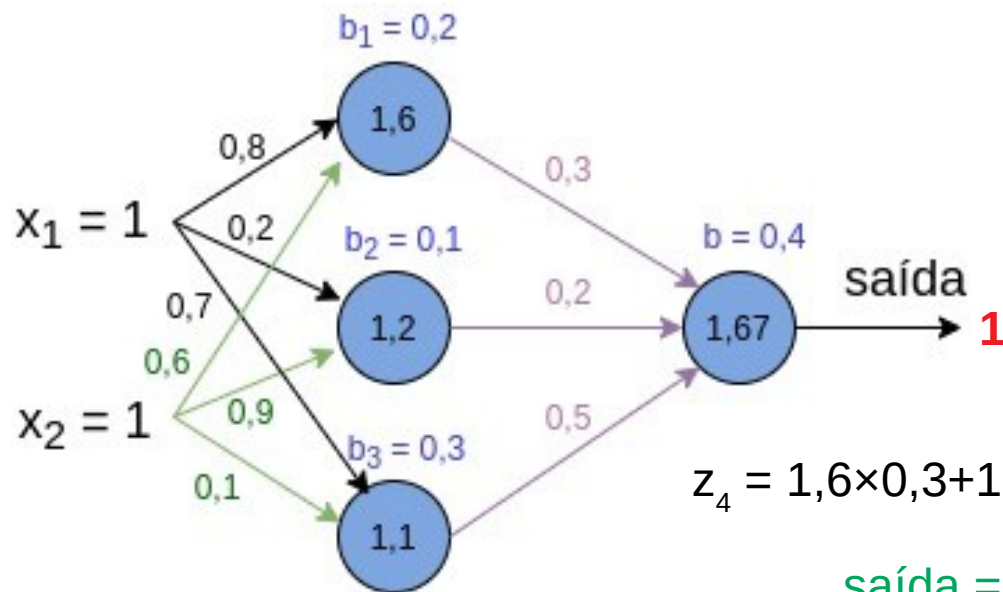
$$z = w^T x + b$$

$$z_1 = 1 \cdot 0,8 + 1 \cdot 0,6 + 0,2 = 1,6$$

$$z_2 = 1 \cdot 0,2 + 1 \cdot 0,9 + 0,1 = 1,2$$

$$z_3 = 1 \cdot 0,7 + 1 \cdot 0,1 + 0,3 = 1,1$$

Multilayer Perceptron



$$z = w^T x + b$$

$$z_1 = 1 \cdot 0,8 + 1 \cdot 0,6 + 0,2 = 1,6$$

$$z_2 = 1 \cdot 0,2 + 1 \cdot 0,9 + 0,1 = 1,2$$

$$z_3 = 1 \cdot 0,7 + 1 \cdot 0,1 + 0,3 = 1,1$$

$$z = w^T x + b > 0$$

$$z_4 = 1,6 \times 0,3 + 1,2 \times 0,2 + 1,1 \times 0,5 + 0,4 = 1,67$$

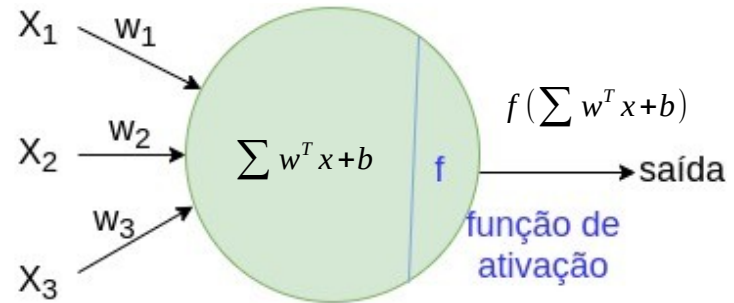
saída = 1 (sem a função de ativação)



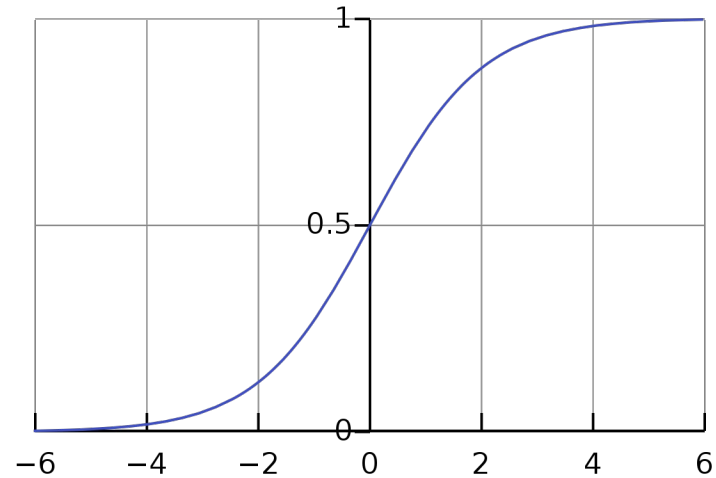
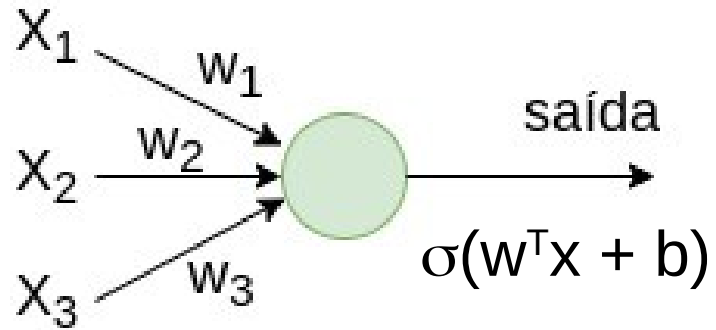
Função de Ativação

Função de Ativação

- ◆ Cada neurônio é caracterizado pelo peso, bias (viés) e a função de ativação
- ◆ Os neurônios fazem uma transformação linear na entrada pelos pesos e bias
- ◆ A função de ativação faz uma transformação não linear

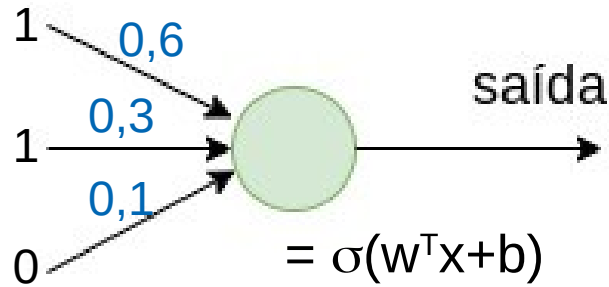


Função Sigmóide



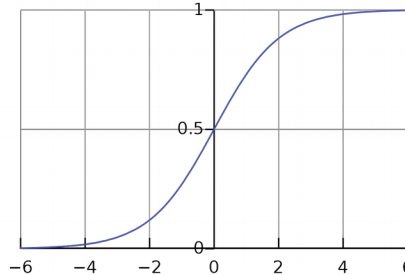
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Função Sigmóide



$$\begin{aligned} &= \sigma(w^T x + b) \\ &= \sigma(1 \cdot 0,6 + 1 \cdot 0,3 + 0 \cdot 0,1 + 0) \\ &= \sigma(0,9) \\ &= 0,7109 \end{aligned}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Você joga tênis hoje?

$x_1 = 1$ (tempo bom)

$w_1 = 0,6$

$x_2 = 1$ (está ventando)

$w_2 = 0,3$

$x_3 = 0$ (estou de bom humor)

$w_3 = 0,1$

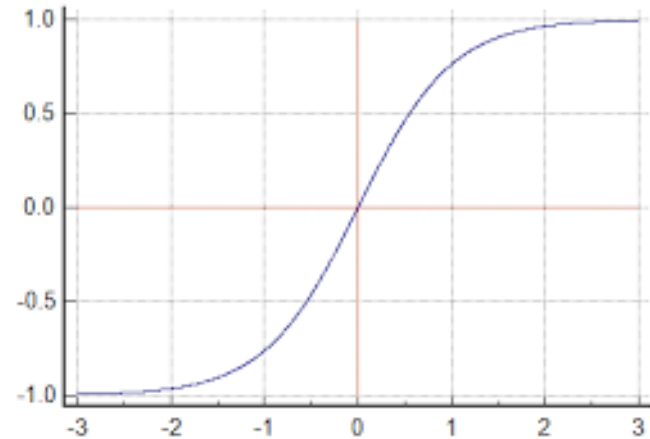
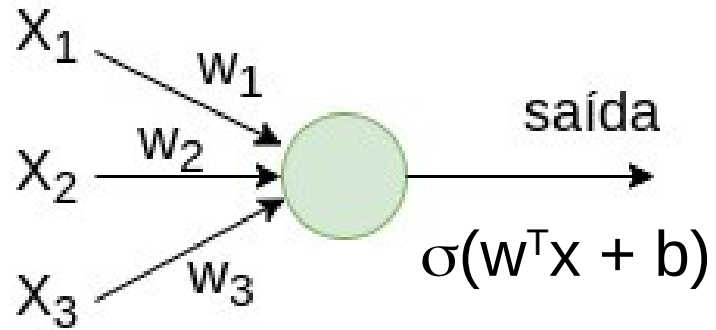
$b = 0$

Função Sigmóide



- ◆ Usada, principalmente, na camada de saída para problemas de classificação binária
- ◆ Apresenta problemas quando os gradientes se tornam muito pequenos
- ◆ Valores entre 0 e 1, função não é simétrica em torno da origem. Nem sempre é interessante enviar somente valores positivos

Função Tanh (Tangente Hiperbólica)



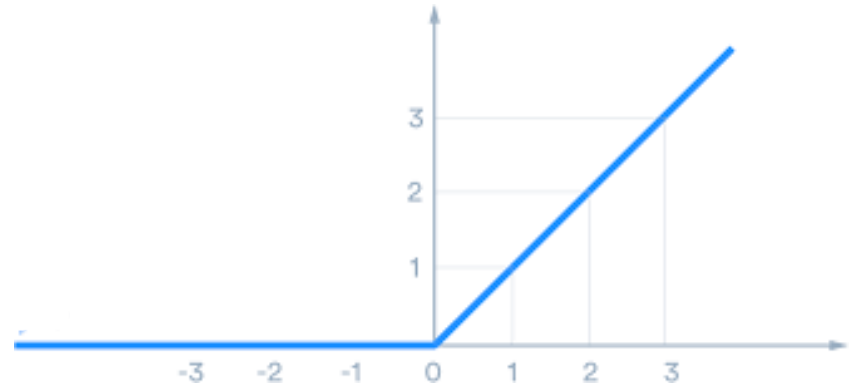
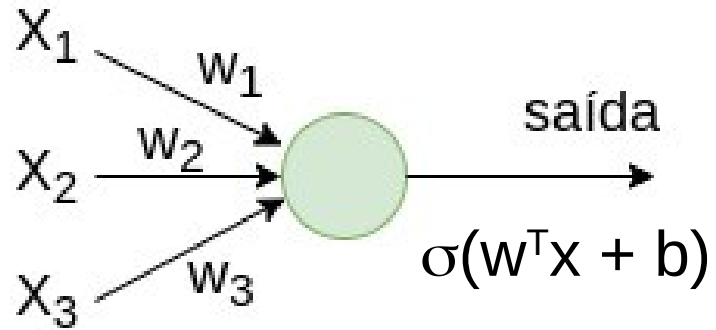
$$\sigma(z) = \frac{2}{1 + e^{-2z}} - 1$$

Função Tanh (Tangente Hiperbólica)



- ◆ Versão escalonada da função Sigmóide
- ◆ Simétrica em relação à origem, valores variam entre -1 e 1

Função ReLU (*Rectified Linear Unit*)



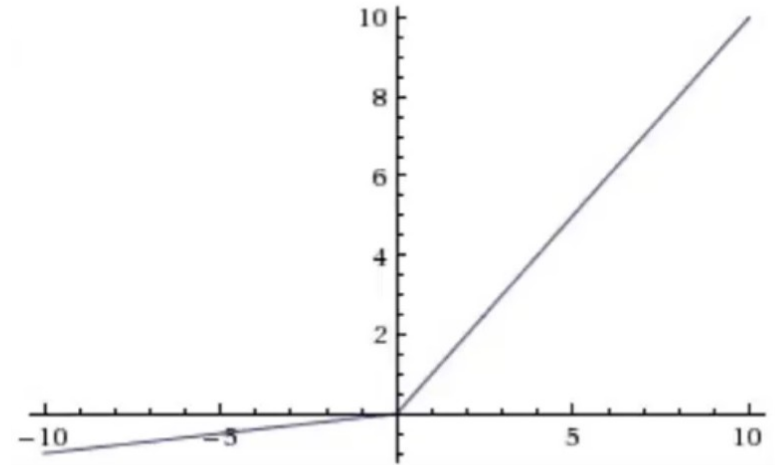
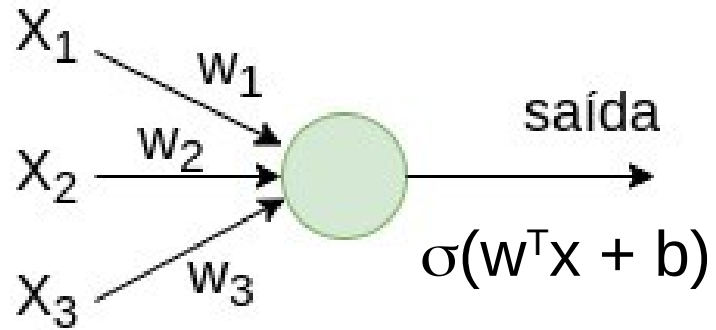
$$\sigma(z) = \max(0, z)$$

Função ReLU (*Rectified Linear Unit*)



- ◆ É a função mais amplamente utilizada
- ◆ Não ativa todos os neurônios ao mesmo tempo, o que torna a rede esparsa e a computação se torna fácil e eficiente
- ◆ Apresenta problemas quando os gradientes se tornam muito pequenos (próximos de zero)

Função Leaky ReLU



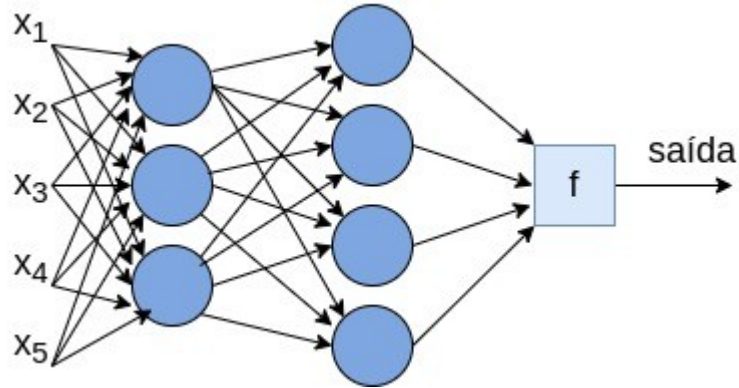
$$\sigma(z) = \begin{cases} az & \text{se } z < 0 \\ z & \text{se } z \geq 0 \end{cases}$$

Função Leaky ReLU



- ◆ Versão melhorada da função ReLU
- ◆ Substitui a linha horizontal por uma linha não-zero, não horizontal
 - ◆ Valor pequeno, como $a = 0,01$
- ◆ Remove o gradiente zero

Função Softmax (ou softargmax)



$$\sigma(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \text{ for } j = 1, \dots, K$$

Função Softmax (ou softargmax)



- ◆ É um tipo de função sigmóide
- ◆ Usada em problemas de classificação multiclasse
 - ◆ Sigmóide é usada para classificação binária
- ◆ Retorna a probabilidade da entrada estar em uma determinada classe
- ◆ Usada na camada de saída do classificador
 - ◆ As outras funções são usadas nas camadas ocultas



Descida do Gradiente

Função de Custo



- ◆ Seja $\mathbf{y}^{(i)}$ o valor da saída correta da rede para cada entrada $\mathbf{x}^{(i)}$, indicada pelos dados de treinamento (m entradas)
- ◆ O algoritmo da rede deve encontrar os pesos ($\mathbf{w}$) e os *bias* ($\mathbf{b}$) para que cada saída $\hat{\mathbf{y}}^{(i)}$ retornada pela rede se aproxime de $\mathbf{y}^{(i)}$
- ◆ A **função de custo** quantifica o quão bem esse objetivo está sendo alcançado

Função de Custo Quadrático



$$C(w, b) = \frac{1}{m} \sum_{i=1}^m \|y^{(i)} - \hat{y}^{(i)}\|^2$$

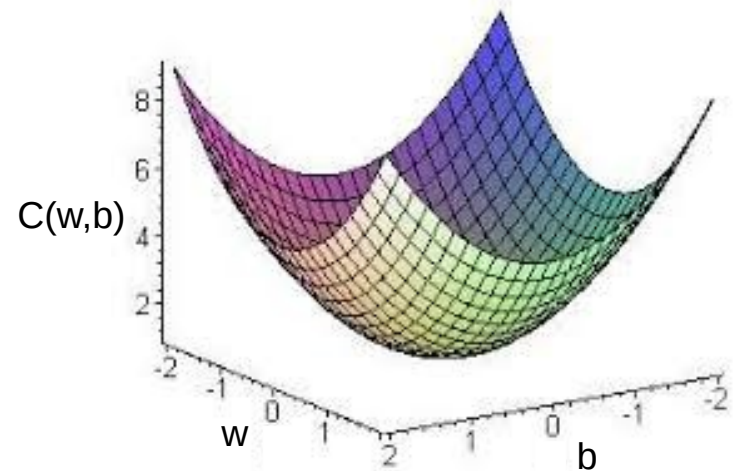
- C é a função de custo quadrático ou erro quadrático médio (MSE - *Mean Squared Error*)
- $\|v\|$ é a norma do vetor v
- $C(w, b) \approx 0$ quando y se aproxima da saída $\hat{y}$
- Objetivo: encontrar pesos e bias que minimizem a função de custo

Descida do Gradiente

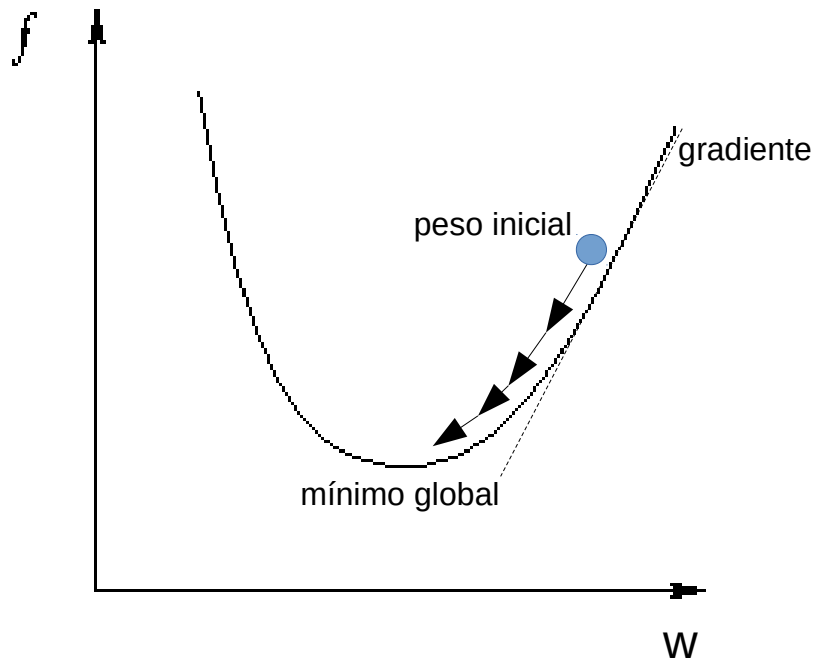
A função de custo é uma função de várias variáveis

A descida do gradiente é um método para otimizar funções complexas (de várias variáveis)

Objetivo: encontrar o valor mínimo (global) da função



Descida do Gradiente



Usa-se derivadas da função de custo para simular a bola rolando

Derivada é a inclinação da função em determinado ponto

Gradiente é um vetor de derivadas

Descida do Gradiente

Inicialize os pesos w e b

Repita (com os valores de pesos atuais)

Passo para frente {

- ◆ passe as entradas do treinamento pela rede
- ◆ calcule a saída (predição)

Passo para trás {

- ◆ compare a saída com a resposta correta
- ◆ calcule o erro
- ◆ retroaja esse erro (*backpropagation*), ajustando os pesos w e b dos neurônios em cada camada

taxa de aprendizagem

derivada da função de custo

$$w \leftarrow w - (\alpha * \text{derivada})$$

Até conseguir o custo zero (ou próximo dele)



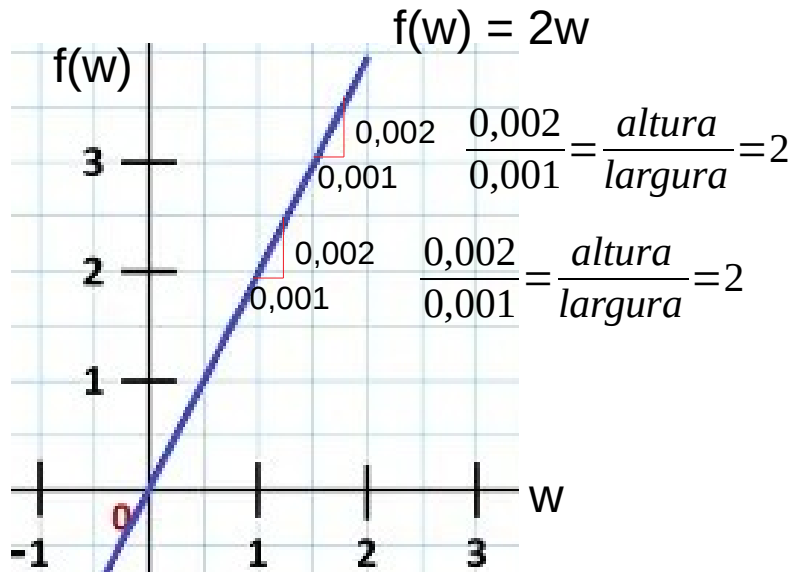
Backpropagation

Backpropagation



- ◆ *Backpropagation* é o nome de um algoritmo usado para calcular o gradiente da função de custo
- ◆ No passo para trás (*backward pass*)
 - ◆ calcule o gradiente da função de perda na camada final
 - ◆ aplique recursivamente a regra da cadeia (*chain rule*) para atualizar os pesos

Derivada



Derivada é a inclinação da função em determinado ponto

$$w = 1,5 \quad f(w) = 3$$

$$w = 1,501 \quad f(w) = 3,002$$

inclinação (derivada) de $f(w)$ no ponto $w = 1,5$ é 2

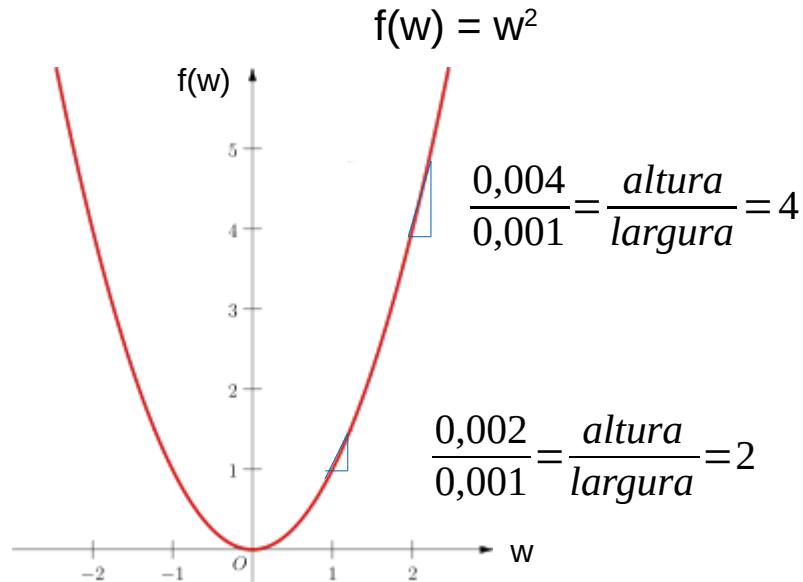
$$w = 1 \quad f(w) = 2$$

$$w = 1,001 \quad f(w) = 2,002$$

inclinação de $f(w)$ no ponto $w = 1$ é 2

$$\frac{df(w)}{dw} = \frac{d}{dw} f(w) = 2$$

Derivada



Derivada é a inclinação da função em determinado ponto

$$w = 2 \quad f(w) = 4$$

$$w = 2,001 \quad f(w) = 4,004$$

inclinação (derivada) de $f(w)$ no ponto $w = 2$ é 4

$$w = 1 \quad f(w) = 1$$

$$w = 1,001 \quad f(w) = 2,002$$

inclinação de $f(w)$ no ponto $w = 1$ é 2

$$\frac{df(w)}{dw} = \frac{d}{dw} f(w) = 2w$$

Grafo Computacional

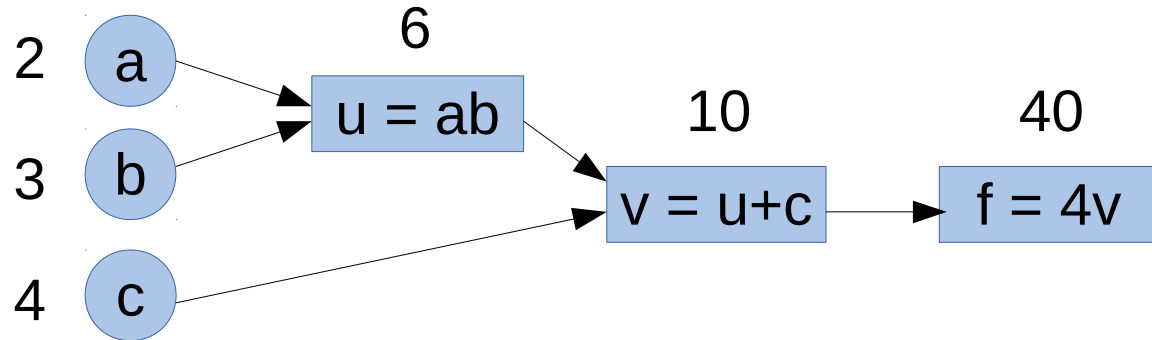
$$f(a,b,c) = 4(\underbrace{ab}_{\underbrace{u}_{\underbrace{v}_{f}}}) + c$$

$$f = 4(2 \cdot 3 + 4) = 40$$

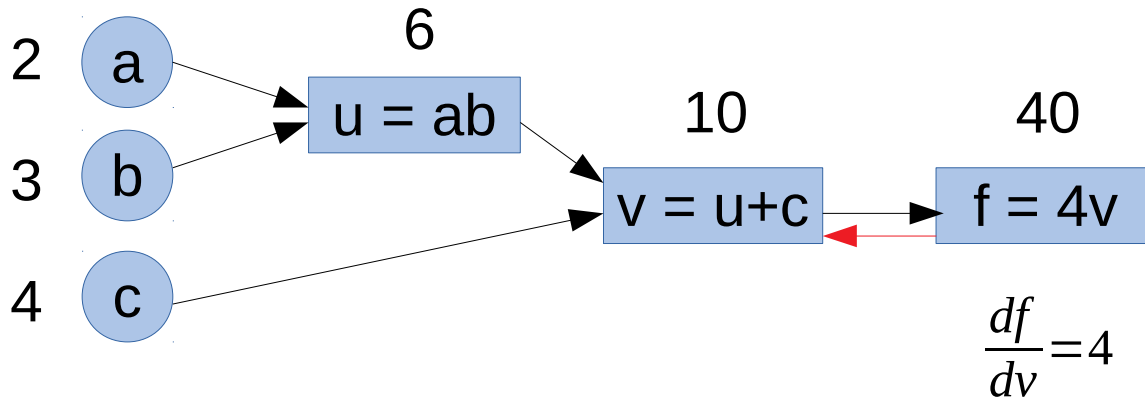
$$u = ab$$

$$v = u + c$$

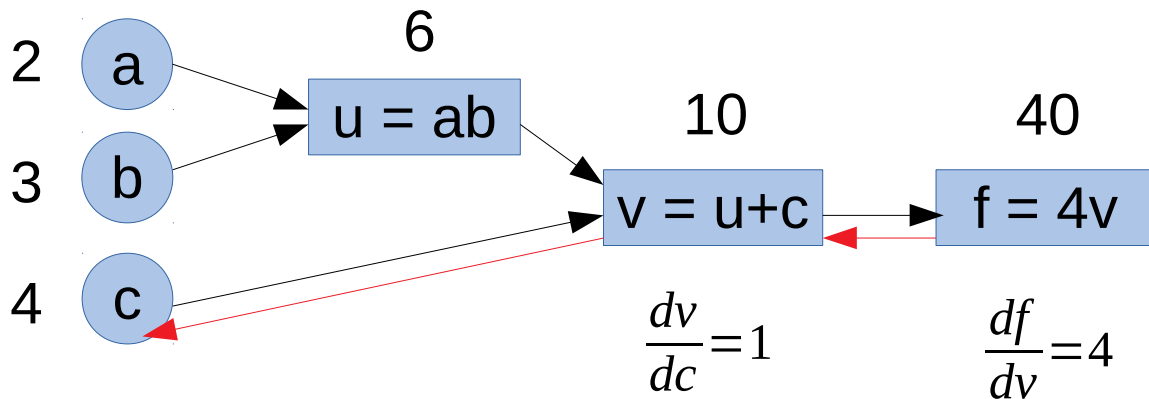
$$f = 4v$$



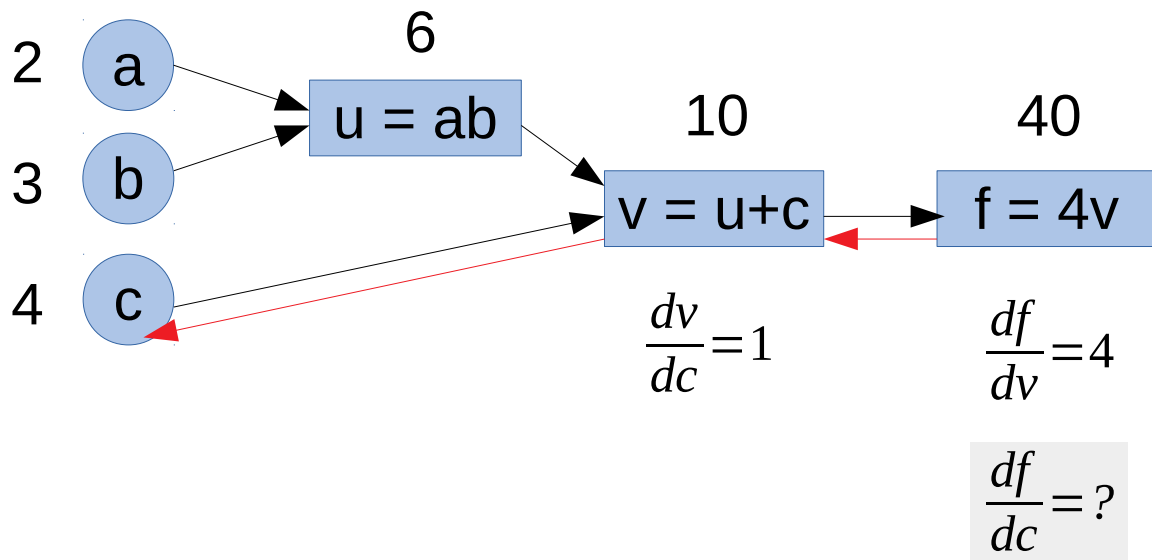
Derivadas em um Grafo Computacional



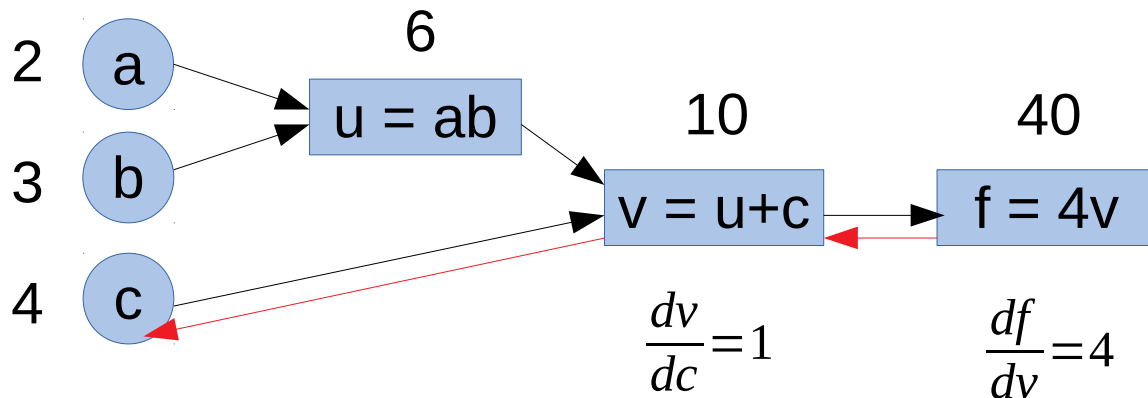
Derivadas em um Grafo Computacional



Derivadas em um Grafo Computacional



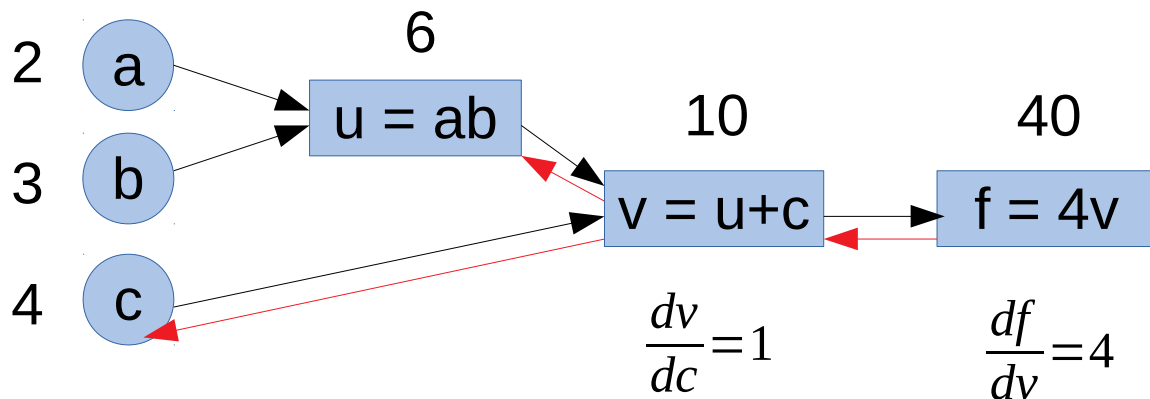
Derivadas em um Grafo Computacional



$$\frac{df}{dc} = \frac{df}{dv} \frac{dv}{dc} = 4$$

Regra da Cadeia

Derivadas em um Grafo Computacional



$$\frac{dv}{dc}=1$$

$$\frac{df}{dv}=4$$

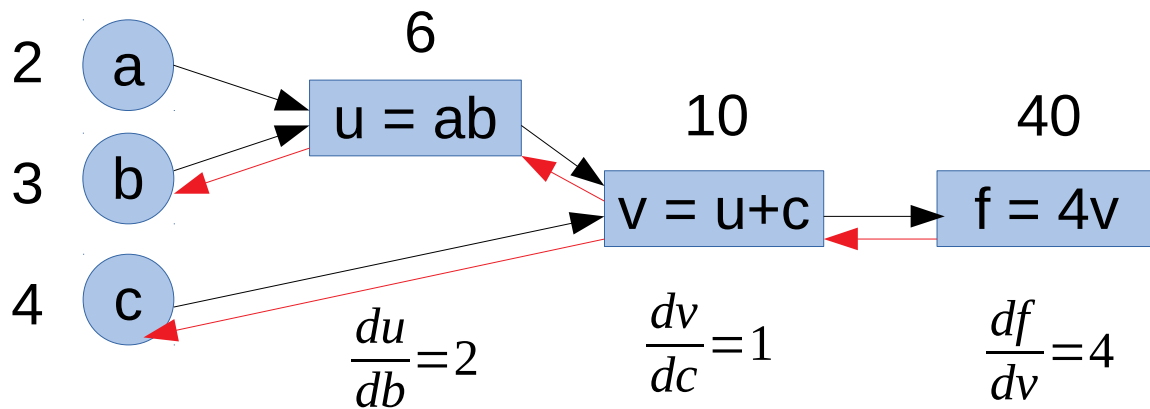
$$\frac{dv}{du}=1$$

$$\frac{df}{dc} = \frac{df}{dv} \frac{dv}{dc} = 4$$

$$\frac{df}{du} = \frac{df}{dv} \frac{dv}{du} = 4$$

Regra da Cadeia

Derivadas em um Grafo Computacional



$$\frac{df}{db} = \frac{df}{du} \frac{du}{db} = 8$$

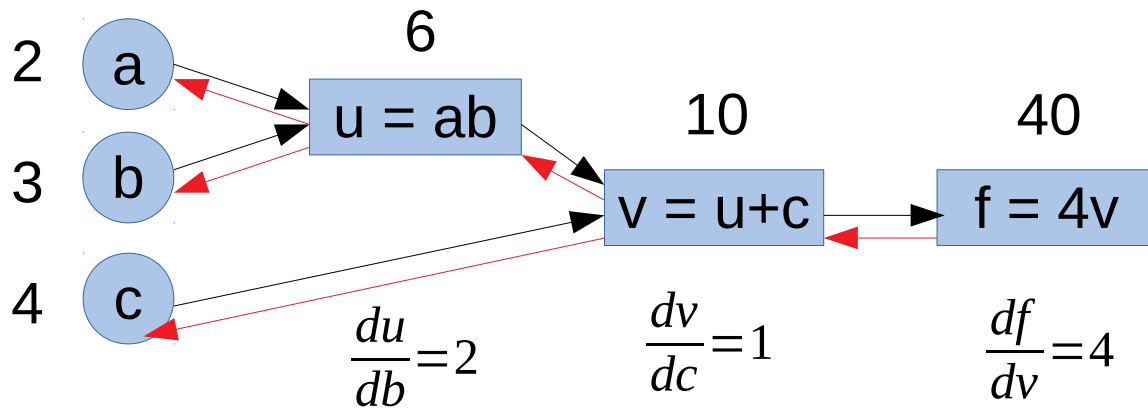
$$\frac{dv}{du} = 1$$

$$\frac{df}{dc} = \frac{df}{dv} \frac{dv}{dc} = 4$$

$$\frac{df}{du} = \frac{df}{dv} \frac{dv}{du} = 4$$

Regra da Cadeia

Derivadas em um Grafo Computacional



$$\frac{df}{db} = \frac{df}{du} \frac{du}{db} = 8$$

$$\frac{df}{da} = \frac{df}{du} \frac{du}{da} = 12$$

$$\frac{du}{da} = 3$$

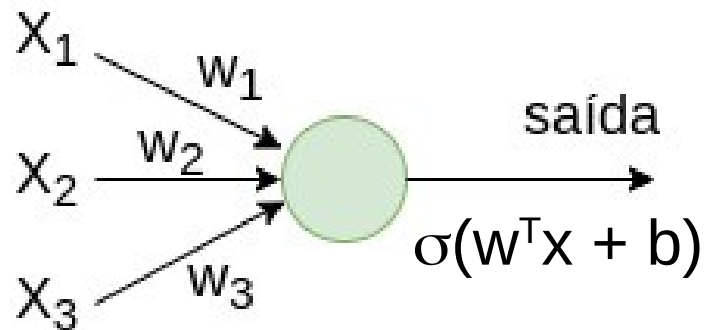
$$\frac{dv}{du} = 1$$

$$\frac{df}{dc} = \frac{df}{dv} \frac{dv}{dc} = 4$$

$$\frac{df}{du} = \frac{df}{dv} \frac{dv}{du} = 4$$

Regra da Cadeia

Função de Perda *Cross-Entropy*



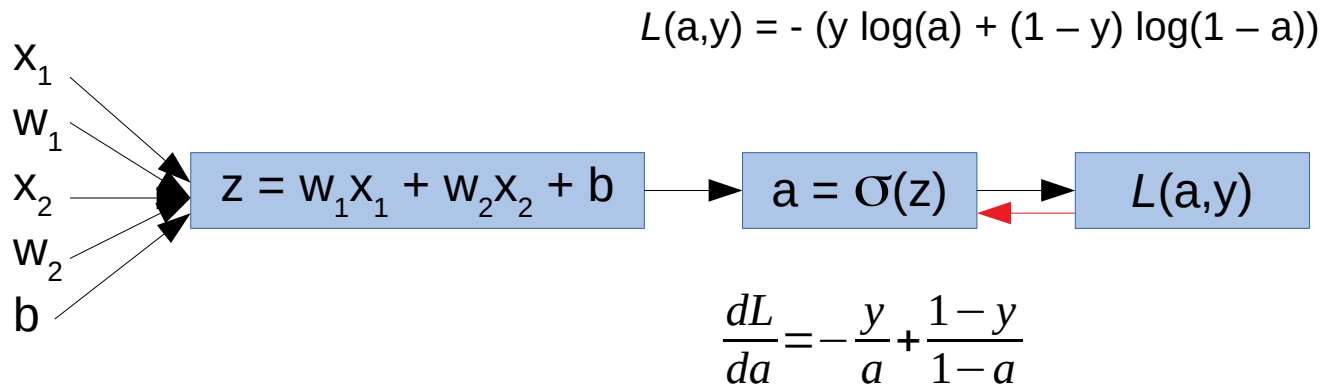
$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

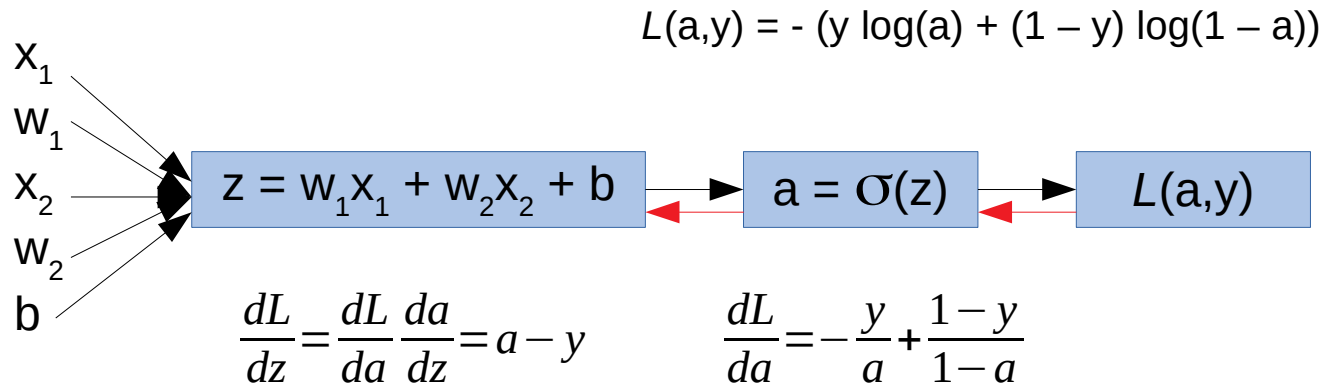
$$L(a, y) = - (y \log(a) + (1 - y) \log(1 - a))$$

Função de perda para Regressão Logística (*Logistic Regression*)

Derivadas para a Função de Perda

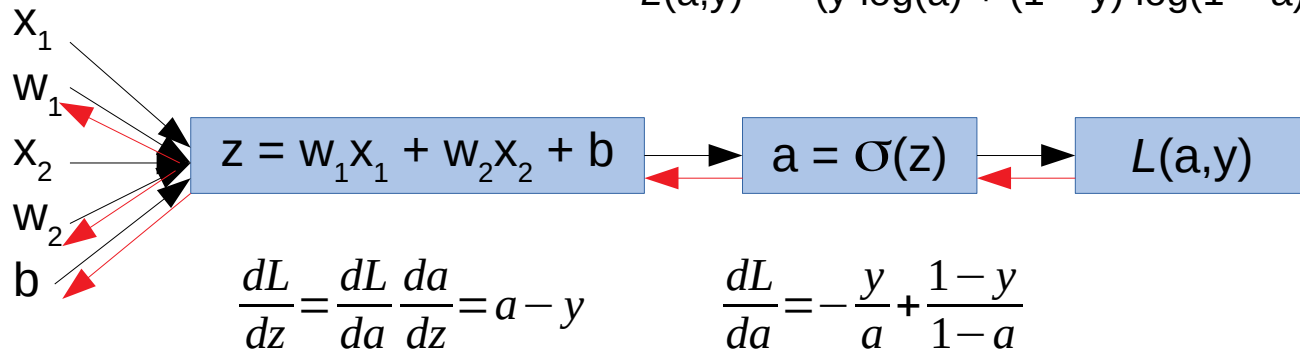


Derivadas para a Função de Perda



Derivadas para a Função de Perda

$$L(a,y) = -(y \log(a) + (1 - y) \log(1 - a))$$

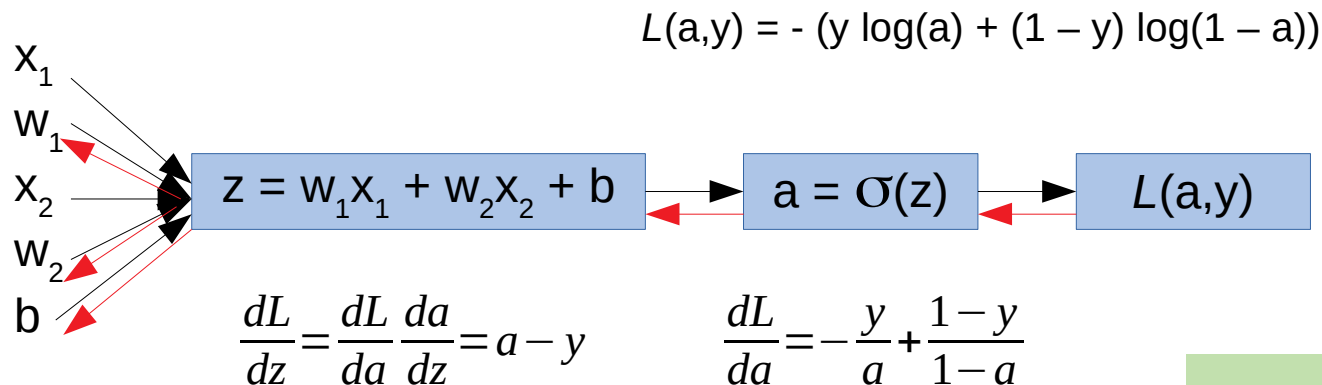


$$\frac{dL}{dw_1} = x_1 \frac{dL}{dz}$$

$$\frac{dL}{dw_2} = x_2 \frac{dL}{dz}$$

$$\frac{dL}{db} = \frac{dL}{dz}$$

Derivadas para a Função de Perda



$$\frac{dL}{dw_1} = x_1 \frac{dL}{dz}$$

$$\frac{dL}{dw_2} = x_2 \frac{dL}{dz}$$

$$\frac{dL}{db} = \frac{dL}{dz}$$

$$w_1 \leftarrow w_1 - \alpha \frac{dL}{dw_1}$$

$$w_2 \leftarrow w_2 - \alpha \frac{dL}{dw_2}$$

$$b \leftarrow b - \alpha \frac{dL}{db}$$

α = taxa de aprendizagem

Função de Custo

$$C(w, b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)} - y^{(i)})$$

função de custo para m
exemplos de treinamento

Calcule a média para cada peso e *bias*

$$\frac{d}{dw_1} C(w, b) = \frac{1}{m} \sum_{i=1}^m \frac{d}{dw_1} L(a^{(i)} - y^{(i)})$$

$$w_1 \leftarrow w_1 - \alpha \frac{dC}{dw_1}$$

Idem para $w_2, w_3, \dots, b_1, b_2, \dots$

Descida do Gradiente

Inicialize os pesos w e b

Repita (com os valores de pesos atuais)

Passo
para
frente



- ♦ passe as entradas do treinamento pela rede
- ♦ calcule a saída (predição)

Passo
para
trás



- ♦ compare a saída com a resposta correta
- ♦ calcule o erro
- ♦ retroaja esse erro (*backpropagation*), ajustando os pesos w e b dos neurônios em cada camada

$$w \leftarrow w - \alpha \frac{dC}{dw}$$

$$b \leftarrow b - \alpha \frac{dC}{db}$$

Até conseguir o custo zero (ou próximo dele)

Implementação em Python



Biblioteca de código aberto para computação numérica usando grafos de fluxos de dados



API para redes neurais de alto nível

Roda sobre o TensorFlow, CNTK, ou Theano

Implementação em Python



Biblioteca de código aberto usada para aplicações tais como visão computacional e processamento de linguagem natural

Referências Bibliográficas



1. Sandro Skansi. Introduction to Deep Learning: From Logical Calculus to Artificial Intelligence. Springer, 2018.

<https://doi.org/10.1007/978-3-319-73004-2>

2. Deep Learning Book. Data Science Academy.

<http://deeplearningbook.com.br/>

Links Interessantes



- <https://cs231n.github.io/neural-networks-1/>
- <https://stevenmiller888.github.io/mind-how-to-build-a-neural-network/>
- <http://neuralnetworksanddeeplearning.com/chap1.html>
- <https://medium.com/onfido-tech/machine-learning-101-be2e0a86c96a>



Link da apresentação:

<https://sites.google.com/ufla.br/denilsonpereira/home/courses/redes-neurais-e-deep-learning?authuser=0>

denilsonpereira@ufla.br

<https://sites.google.com/ufla.br/denilsonpereira/>



Este trabalho está licenciado com uma Licença
[Creative Commons - Atribuição-NãoComercial-SemDerivações 4.0](https://creativecommons.org/licenses/by-nc-nd/4.0/)